Python Code

```python
import time
from itertools import combinations
from collections import defaultdict

import networkx as nx
import community as community_louvain  # python-louvain


# ============================================================
# 1. LOAD HYPERGRAPH
# ============================================================
def load_hypergraph(file_path):
    hyperedges = []
    with open(file_path, 'r') as f:
        for line in f:
            nodes = list(map(int, line.strip().split(',')))
            if len(nodes) > 1:
                hyperedges.append(nodes)
    return hyperedges


# ============================================================
# 2. HYPERGRAPH → GRAPH (CLIQUE EXPANSION)
# ============================================================
def hypergraph_to_graph(hyperedges):
    G = nx.Graph()

    for hedge in hyperedges:
        for u, v in combinations(hedge, 2):
            if G.has_edge(u, v):
                G[u][v]['weight'] += 1
            else:
                G.add_edge(u, v, weight=1)

    return G


# ============================================================
# 3. COMMUNITY DETECTION (LOUVAIN)
# ============================================================
def detect_communities(G):
    partition = community_louvain.best_partition(G, weight='weight')

    communities = defaultdict(list)
    for node, comm in partition.items():
        communities[comm].append(node)

    return list(communities.values()), partition


```

```python
# ============================================================
# 4. STANDARD MODULARITY
# ============================================================
def compute_modularity(G, partition):
    return community_louvain.modularity(partition, G, weight='weight')


# ============================================================
# 5. GRAPH → ADJACENCY LIST
# ============================================================
def graph_to_adj(G):
    adj = defaultdict(set)
    for u, v in G.edges():
        adj[u].add(v)
        adj[v].add(u)
    return adj


# ============================================================
# 6. PARTITION → COMMUNITY LIST
# ============================================================
def partition_to_communities(partition):
    comms = defaultdict(list)
    for node, c in partition.items():
        comms[c].append(node)
    return list(comms.values())


# ============================================================
# 7. FACULTY EXTENDED MODULARITY
# ============================================================
def extended_modularity_faculty(adj, communities):
    deg = {u: len(adj[u]) for u in adj}
    m2 = sum(deg.values())

    if m2 == 0:
        return 0.0

    # count how many communities each node belongs to
    n_comm = defaultdict(int)
    for comm in communities:
        for node in comm:
            n_comm[node] += 1

    total = 0.0

    for U in communities:
        for i, j in combinations(U, 2):
            A = 1 if j in adj.get(i, set()) else 0

            expected = (deg.get(i, 0) * deg.get(j, 0)) / m2

            total += 2 * ((A - expected) / (n_comm[i] * n_comm[j]))

```

```python
    return total / m2


# ============================================================
# 8. FACULTY OVERLAPPING MODULARITY
# ============================================================
def overlapping_modularity_faculty(adj, communities):
    deg = {u: len(adj[u]) for u in adj}
    N = sum(deg.values())

    if N == 0:
        return 0.0

    msum = 0.0

    for U in communities:
        Uset = set(U)

        def Sdeg(x):
            return len(adj[x] & Uset)

        # nodes appearing in multiple communities
        others = set().union(*[set(k) for k in communities if k is not U]) if len(communities) > 1 else set()
        ov = Uset & others

        for i_idx in range(len(U)):
            ui = U[i_idx]

            for j_idx in range(i_idx + 1, len(U)):
                uj = U[j_idx]

                al1 = 1.0
                al2 = 1.0

                # weight for overlapping nodes
                if ui in ov:
                    o = Sdeg(ui)
                    o1 = sum(len(adj[ui] & set(ll)) for ll in communities if ui in ll)
                    al1 = (o / o1) if o1 else 0

                if uj in ov:
                    oo = Sdeg(uj)
                    oo1 = sum(len(adj[uj] & set(ll)) for ll in communities if uj in ll)
                    al2 = (oo / oo1) if oo1 else 0

                has_edge = 1 if ui in adj.get(uj, set()) else 0

                expected = (deg.get(ui, 0) * deg.get(uj, 0)) / (2 * N)

                x = (has_edge - expected) * al1 * al2

```

```python
                msum += 2 * x

    return msum / (2 * N)


# ============================================================
# 9. FULL PIPELINE
# ============================================================
def run_pipeline(file_path):
    start_time = time.time()

    # Step 1: Load hypergraph
    hyperedges = load_hypergraph(file_path)

    # Step 2: Convert to graph
    G = hypergraph_to_graph(hyperedges)

    # Step 3: Community detection
    communities, partition = detect_communities(G)

    # Step 4: Convert formats
    adj = graph_to_adj(G)
    comm_list = partition_to_communities(partition)

    # Step 5: Metrics
    modularity = compute_modularity(G, partition)
    ext_mod = extended_modularity_faculty(adj, comm_list)
    overlap_mod = overlapping_modularity_faculty(adj, comm_list)

    end_time = time.time()

    # Step 6: Output
    print("\n==============================")
    print(f"Dataset: {file_path}")
    print(f"Communities Found: {len(comm_list)}")
    print(f"Standard Modularity: {modularity:.4f}")
    print(f"Extended Modularity (Faculty): {ext_mod:.4f}")
    print(f"Overlapping Modularity (Faculty): {overlap_mod:.4f}")
    print(f"Execution Time: {end_time - start_time:.4f} seconds")
    print("==============================\n")


# ============================================================
# 10. RUN ALL DATASETS
# ============================================================
if __name__ == "__main__":

    datasets = [
        "apr6_lab10/hyperedges-contact-high-school.txt",
        "apr6_lab10/hyperedges-contact-primary-school.txt",
        "apr6_lab10/hyperedges-house-bills.txt",
        "apr6_lab10/hyperedges-house-committees.txt",
        "apr6_lab10/hyperedges-senate-bills.txt",
        "apr6_lab10/hyperedges-senate-committees.txt",
```

```python
        "apr6_lab10/hyperedges-trivago-clicks.txt",
        "apr6_lab10/hyperedges-walmart-trips.txt",
        "apr6_lab10/linear_large_edges_1000_he.txt",
    ]

    for dataset in datasets:
        run_pipeline(dataset)
```